
Exploring Non-Local Effects Of Parameter Updates in Deep Q-Learning

Kavya Anbarasu
kavyaa@mit.edu

Puja Balaji
kerberos@mit.edu

Carol Jiang
cjiang24.edu

Kairo Morton
mortonk@mit.edu

Abstract

In this paper, we explore the influence of parameter updates in reinforcement learning algorithms on both local and non-local state-action pairs, using OpenAI's FrozenLake-v1 environment. We use basic tabular Q learning and deep Q-learning (DQN) to analyze the impact of neural network modifications, including activation functions, input representations, and memory utilization strategies. To quantify the extent of learning generalization across different state-action pairs, we use mean absolute error (MAE) between pre- and post-update Q-values and an optimality score (OS) comparing update directions to theoretical ideals. Greater understanding of the non-local effects of weight changes in a neural network can help to assess and improve the scalability and stability of these methods. Next steps will include implementing Proximal Policy Optimization to compare its performance with Q-learning methods and to further assess the influence of policy changes on non-local states. Additionally, experiments with non-sparse reward environments, such as Taxi-v3, will help to understand the effects of more frequent and varied feedback on network learning dynamics. Systematic variation in reinforcement learning parameters and architectures will also be explored to determine the sensitivity and adaptability of learning algorithms to different configurations.

1 Introduction

We aim to investigate how parameter changes to the policy in one state affect changes in distant states and observe how this behavior might affect learning and generalization of the system. The motivation behind our project lies in understanding the non-local effects of weight changes in a neural network, which then helps assess the scalability and stability of these methods. To start, we apply tabular Q learning in OpenAI's FrozenLake-v1 environment. This environment is simple and computationally inexpensive, with sparse reward and a small discrete state space, suitable features for our purposes. Tabular Q-learning provides a clear baseline for the following experiments.

We then move to a deep-learning approach by applying a neural network-based Q-learning algorithm to the same agent-environment setup in order to analyze the effects of the neural network updates, and compare how the state-action pairs were affected in the two approaches. To build on top of the deep Q-learning approach, we run additional experiments changing the activation function of the neural network from the conventional ReLU function, changing the input representation to (x, y) coordinates to capture spatial dependencies more explicitly, and changing the replay buffer and batch sizes to explore their impacts on memory utilization and sampling strategies. For each of these, we record the mean change in Q-values for in-batch states and for out-of-batch states to better understand how each of these alterations affect local and non-local states. Additionally, we define metrics to evaluate the optimality of the updates made by the agent within and out of the batch. Analyzing and understanding

this behavior results in valuable insights on learning dynamics and system generalization with the potential to exploit locality or non-local Q-value changes for optimization.

2 Experiment Setup and Training Environment

We used OpenAI’s gym FrozenLake-v1, a sparse reward environment, which is an 8x8 grid consisting of 64 states and 4 possible actions (left, down, up, and right). The grid consists of safe, starting point S, safe, frozen surface F, unsafe hole H, and safe goal G.

The base learning algorithms we will leverage are tabular and deep Q-learning (DQN). For these methods, we will track metrics during training related to Q-values changes to understand how network parameter updates impact the Q-values of in-batch and out-of-batch states/state-action pairs.

The update function for Q-learning is

$$Q(s, a) = Q(s, a) + \alpha * (r(s, a) + \gamma * \max_a Q(s', a) - Q(s, a)) \quad (1)$$

and the temporal difference loss function in DQN is defined as

$$L(\theta) = (Q_\theta(s, a) - (r + \gamma \max_{a'} \hat{Q}(s', a')))^2 \quad (2)$$

where s , s' , r , and a denote the current state, next state, reward, and action in both equations whereas Q_θ and $\hat{Q}$ represent the regular and target Q networks from the traditional DQN formulation respectively.

We used a multi-layer perceptron Q-network for our baseline deep learning approach. The network architecture is composed of a linear layer followed by the ReLU activation function, with one hidden layer of the same form and a final linear layer. The forward pass takes an input tensor representing the (discrete) state observation, and converts it into one-hot encoded vectors, before passing it through the network to generate Q-values for each action.

To answer our question of whether parameter changes affect distant states, we record two distinct metrics separately for in-batch and out-of-batch state-action pairs during training. First, we record the mean absolute error (MAE) between Q-values before and after a gradient update step. This metric aims to determine if there are differences in the magnitude of Q-value updates for in-batch and out-of-batch state-action pairs. Higher values of the mean absolute difference in Q-values for out-of-batch states may indicate that the agent’s learning is not only confined to the sampled experiences but also extends to the broader state-action space, suggesting that the agent’s learning is generalizing well beyond the sampled experiences in the batch. The metric is defined mathematically as follows:

$$\text{MAE}(\theta, \theta', P) = \frac{1}{|P|} \sum_{(s,a) \in P} |Q_{\theta'}(s, a) - Q_\theta(s, a)| \quad (3)$$

where θ and θ' are the Q-network parameters before and after one gradient descent step respectively and P denotes either the set of in-batch or out-of-batch state-action pairs.

Second, we compute the optimality of Q-value changes made by the DQN agent for in-batch and out-of-batch (state, action) pairs, assessing the cosine similarity between the actual update directions of state-action pairs and the theoretically optimal update directions (defined by the update vector pointing towards the converged values). A higher value of this optimality score (OS) for out-of-batch states would indicate that the Q-value updates made outside the batch are in the direction of theoretically optimal updates, also demonstrating the agent’s generalization capabilities. This optimality score metric (OS) can be defined as

$$\text{OS}(\theta, \theta', \theta^*, P) = \frac{1}{Z} \sum_{(s,a) \in P} (Q_{\theta^*}(s, a) - Q_\theta(s, a))(Q_{\theta'}(s, a) - Q_\theta(s, a)) \quad (4)$$

where θ^* are the parameters of the converged final Q-network and P denotes either the set of in-batch or out-of-batch state-action pairs. Additionally, the normalization factor Z denotes the product of the magnitudes of the two update vectors following the traditional computation cosine similarity.

3 Experiments

3.1 Tabular Q-learning

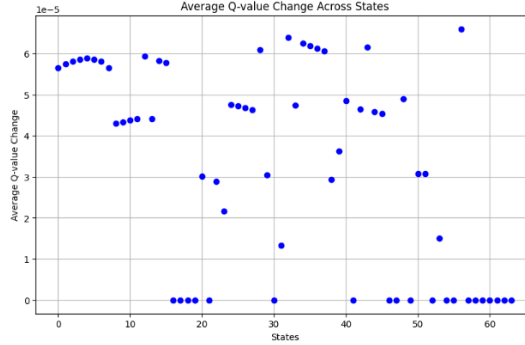


Figure 1: Average Q-value Change for different runs

In tabular Q-learning, each state-action pair has its own Q-value that is based on immediate rewards and the Q-values of next states. There are no non-local effects because Q-values are updated independently for each state-action pair. As a baseline, when initializing the q-values to zero, the average change in q-value for each state throughout a run ranges from 0 to $7e-5$, while the q-values approach 1.

3.2 Deep Q-learning

Our baseline deep Q-learning experiment was run with a ReLU activation function, one-hot encoded input vectors for states, a replay buffer size of 10000, batch size of 128, and on an 8 x 8 map size with deterministic dynamics for 100000 steps. This yielded a mean optimality score of 0.0380 with a variance of 0.1606 for in-batch states, and a mean optimality score of 0.0298 with a variance of 0.1214 for out-of-batch states.

3.3 Activation function

Yang et. al have empirically shown through spectral analysis of the Neural Tangent Kernel that the similarity of gradient vectors for different state-action pairs is activation function dependent when using a Q network with a MLP architecture. This finding led us to ablate the activation function in our experimental setup. We computed both the mean and variance of our two key metrics across DQN training using three different non-linear activation functions: Tanh, Leaky ReLU and Sigmoid. We only report the results for the optimality score metric in Table 1 as we found that the in-batch MAE is only slightly higher than the out-of-batch MAE for all activation functions.

Table 1: Activation Function Impact on Q-value Optimality Score

Activation Function	In-Batch OS (mean/var.)	Out-of-Batch OS (mean/var.)
ReLU (Baseline)	0.0578/0.1740	0.0496/0.1317
Tanh	0.0213/0.1497	0.0092/0.0882
Leaky ReLU	0.0520/0.1941	0.0335/0.1121
Sigmoid (300k steps to converge)	0.1095/0.2148	0.0468/0.1442

3.4 State representation

Here, the state representation was modified to utilize (x, y) coordinates for each position instead of one-hot encoding. This change in state representation aims to maintain a spatial relationship among states that is absent in one-hot encoding, where states are represented as orthogonal vectors with no inherent notion of closeness or distance. By representing states as (x, y) coordinates, our model can potentially learn and exploit the spatial locality of the grid. We tested several configurations of

maintaining spatial locality (the raw (x, y) coordinates, normalizing the coordinates between 0 to 1, using the sine and cosine functions to try to allow the model to learn about circular relationships and continuity at the boundaries, and using positional encodings), but found that only normalizing the values between 0 to 1 yielded a stable result with the good convergence, so that is the only experiment reported in the table. Our experiments show that the in-batch performance increases when maintaining spacial locality in the state while out of batch performance decreases.

Table 2: Maintaining Spatial Locality in Embedding— Impact on Q-Value MAE

Embedding Type/Batch	In-Batch MAE	Out-of-Batch MAE
Baseline	0.0011	0.0009
(x, y) normalized b/w 0 to 1, Batch Size: 128	0.0020	0.0023
(x, y) normalized b/w 0 to 1, Batch Size: 32	0.0017	0.0019

Table 3: Maintaining Spatial Locality in Embedding— Impact on Q-Value Optimality Score

Embedding Type/Batch	In-Batch OS (mean/var.)	Out-of-Batch OS (mean/var.)
Baseline	0.0578/0.1740	0.0496/0.1317
(x, y) normalized b/w 0 to 1, Batch Size: 128	0.0595/0.2105	0.0294/0.1824
(x, y) normalized b/w 0 to 1, Batch Size: 32	0.0618/0.2529	-0.0041/0.1767

3.5 Replay buffer/batch size

As discussed in lecture, the replay buffer and batch size can largely influence the agent’s learning, as training would be less robust to sparse reward with a small buffer size, resulting in less stable learning. Similarly, a small batch size means the updates to the Q-network are based on fewer experience sampled from the replay buffer. We altered the replay buffer and batch sizes to observe the effects on the two metrics described, with results shown below. Each experiment yielded similar in-batch and out-of-batch mean Q-value differences (MAE).

Table 4: Replay Buffer/Batch Size Impact on Q-value MAE

Replay buffer/batch size	In-Batch MAE	Out-of-Batch MAE
Baseline	0.0011	0.0009
Batch size: 32	0.0015	0.0013
Batch size: 1	0.0013	0.0007
Replay buffer size: 128	0.0024	0.0016
Replay buffer/batch size: 32	0.0015	0.0009

Table 5: Replay Buffer/Batch Size Impact on Q-value Optimality Score

Replay buffer/batch size	In-Batch OS (mean/var.)	Out-of-Batch OS (mean/var.)
Baseline	0.0578/0.1740	0.0496/0.1317
Batch size: 32	0.0829/0.2292	0.0390/0.1231
Batch size: 1	0.0625/0.9960	0.0643/0.3441
Replay buffer size: 128	0.0888/0.2195	0.0606/0.1705
Replay buffer/batch size: 32	0.0821/0.3682	0.0818/0.2660

4 Conclusion

We found similar in-batch and out-of-batch Q-value changes in DQN with batch size and state representation contributing the most. We recommend improving upon the metrics we use in future work by exploring the theory behind parameter changes in DQN.

References

[1] Yang, G. Anurag, A., & Agarwal, P. Overcoming the Spectral Bias of Neural Value Approximation. *arXiv*, arXiv:2206.04672.